CLIP-Branches: Interactive Fine-Tuning for Text-Image Retrieval

Christian Lülf
University of Münster
Münster, Germany
christian.luelf@uni-muenster.de

Denis Mayr Lima Martins
Independent Researcher
Campinas, Brazil
denismartins@acm.org

Marcos Antonio Vaz Salles
Independent Researcher
Cascais, Portugal
msalles@acm.org

Yongluan Zhou
University of Copenhagen
Copenhagen, Denmark
zhou@di.ku.dk

Fabian Gieseke
University of Münster
Münster, Germany
fabian.gieseke@uni-muenster.de

ABSTRACT

The advent of text-image models, most notably CLIP, has significantly transformed the landscape of information retrieval. These models enable the fusion of various modalities, such as text and images. One significant outcome of CLIP is its capability to allow users to search for images using text as a query, as well as vice versa. This is achieved via a joint embedding of images and text data that can, for instance, be used to search for similar items. Despite efficient query processing techniques such as approximate nearest neighbor search, the results may lack precision and completeness. We introduce *CLIP-Branches*, a novel text-image search engine built upon the CLIP architecture. Our approach enhances traditional text-image search engines by incorporating an interactive fine-tuning phase, which allows the user to further concretize the search query by iteratively defining positive and negative examples. Our framework involves training a classification model given the additional user feedback and essentially outputs all positively classified instances of the entire data catalog. By building upon recent techniques, this inference phase, however, is not implemented by scanning the entire data catalog, but by employing efficient index structures pre-built for the data. Our results show that the fine-tuned results can improve the initial search outputs in terms of relevance and accuracy while maintaining swift response times.

CCS CONCEPTS

• **Information systems → Search engine architectures and scalability**.

KEYWORDS

text-image retrieval, quantization, relevance feedback, CLIP

ACM Reference Format:
Christian Lülf, Denis Mayr Lima Martins, Marcos Antonio Vaz Salles, Yongluan Zhou, and Fabian Gieseke. 2024. *CLIP-Branches*: Interactive Fine-Tuning for Text-Image Retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24), July 14–18, 2024, Washington, DC, USA.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3626772.3657678

SIGIR '24, July 14–18, 2024, Washington, DC, USA

ACM ISBN 979-8-4007-0431-4/24/07
https://doi.org/10.1145/3626772.3657678

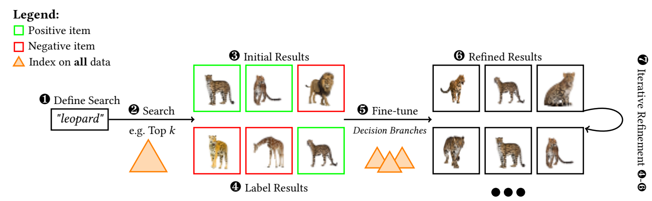


Figure 1: Search process of CLIP-Branches. Traditional text-to-image search engines only consist of the steps ❶ to ❸ while CLIP-Branches adds a fine-tuning stage to refine the initial search results (Steps ❹-❼). Index structures are employed during the search for faster execution.

1 INTRODUCTION

Recent advances in deep learning have led to significant improvements in text-based image retrieval systems. Traditional methods, such as those employed by image platforms like Pinterest or Flickr, typically rely on nearest neighbor (NN) search algorithms for data retrieval [18], where a set of similar items is returned for a given query text or image. These algorithms facilitate data retrieval through text and image embeddings, offering a rapid and efficient solution for handling extensive data sets through the use of appropriate index structures. The resulting answer set, however, might be suboptimal in the sense that desired objects of the data catalog might be missing or that objects not matching the query are given in the answer set. Information retrieval approaches such as learning-to-rank [10] could be used to rank the set of nearest neighbors returned for a given query, e.g., using relevance feedback to increase search precision. However, such fine-tuning steps only consider the results returned by an NN approach, and typically still lead to a significant number of false negatives and positives.

The development of text-image models, particularly CLIP [15], has substantially impacted the realm of text-based image retrieval. CLIP yields a joint embedding space for both text and image inputs and has given rise to various text-image search engines [1, 3]. One significant outcome of CLIP is, among other things, its capability to enable users to search for images using text as a query, as well as vice versa, which is achieved via the joint embedding space. As for the retrieval systems mentioned above, search queries can be implemented using (efficient) nearest neighbor search techniques. Nonetheless, searching for the nearest neighbors (e.g., top k) still often falls short in precision and relevance, i.e., the answer set for a query might contain examples not matching the query and might not contain all relevant examples. This is illustrated in Figure 1

(Steps ❶-❸). Here, the (text) query "leopard" yields an answer set of images. However, some images do not fit well to the query, and many leopard images might still be missing. This issue is further amplified by inherent limitations in text-image models like CLIP, which struggle to achieve fine-grained semantic understanding [5].

In this work, we present CLIP-Branches, a novel text-image search engine that extends traditional text-image search engines by incorporating relevance feedback through an iterative fine-tuning phase based on the concept of fast *search-by-classification* [11]. Search-by-classification allows users to express their query intent by simply providing examples of what they seek (positive instances) and do not seek (negative instances). For example, given the search for "leopard" and the initial answer set, the user can further concretize the search intent by labeling some of the answer images as positive and some as negative, see Figure 1 (Step ❹). These new labels can then be used to train a classification model, which, in turn, can be applied to the entire data catalog to retrieve more positive examples, see Figure 1 (Steps ❺-❼). Instead of scanning the entire data catalog, which would result in high response times,[1] we resort to so-called *decision branches* [11], a recently proposed variant of decision trees, which, in conjunction with pre-built index structures, allows us to efficiently implement the model application phase, leading to response times in the range of seconds only. Note that unlike traditional NN-based search engines that essentially only return (re-ranked) nearest neighbors for a given query, CLIP-Branches operates on the entire (!) database and retrieves all instances in the data catalog that are classified as positive by the model. Hence, this kind of retrieval is particularly beneficial in scenarios where users require a more complete set of query-relevant instances, which are obtained via an iterative and interactive refinement process.

The main contributions of this work are as follows:

(1) We present CLIP-Branches, a novel text-image search engine that enhances traditional models by incorporating user feedback for fine-tuning search results.[2]
(2) Several adaptations are needed to combine CLIP with decision branches. Among other things, we resort to suitable quantization techniques [7, 8] to reduce the storage footprint of the CLIP embeddings. This is achieved by extending the neural network architecture used for CLIP and by adapting some of its weights using suitable loss functions.
(3) We demonstrate the effectiveness of CLIP-Branches across various extensive data sets encompassing over 260 million image instances. We also provide the source code that allows other researchers to (re)use the search engine for their own data and use cases. The code repository is publicly available.[3]

Search-by-classification has already proven effective for geospatial search tasks [12]. CLIP-Branches introduces this method to multimodal text-image retrieval, leveraging CLIP embeddings to merge text and image data within a unified feature space.

2 TECHNICAL OVERVIEW

A key aspect of the functionality of text-image search engines is the rapid response time, a critical requirement given that users typically expect quick access to search results. These search engines predominantly leverage pre-built index structures, such as FAISS [6], to achieve fast search times. Such index structures are generally populated with feature embeddings of the images that have been extracted beforehand, e.g., using deep neural networks.

[1] Applying a decision tree to a large data catalog might easily take minutes or hours.
[2] A prototype is available at: https://web.clip-branches.net/
[3] https://github.com/cluel01/clip-branches

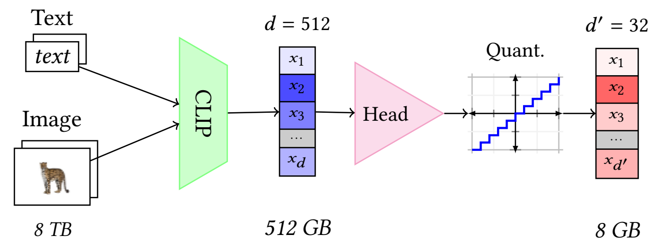


Figure 2: Feature extraction for CLIP-Branches. The initial text and image data are transformed into 512-dimensional embeddings using CLIP. These embeddings are refined via a custom head module, followed by 8-bit quantization, resulting in a 32-dimensional 8-bit embedding. The reduction in data size is exemplified below using the LAION data set.

2.1 Search Workflow

A general workflow of a text-image search engine is depicted in Figure 1. Note that traditional search engines usually only cover Steps ❶-❸. CLIP-Branches extends on that setup by adding a search-by-classification stage (Steps ❹-❼) [11]. After the first results have been retrieved in Step ❸, users can refine their search intent by selecting positive (correctly identified images) and negative (incorrectly identified images) instances (Step ❹). These instances serve as a training set for the decision branches [11] mentioned above. This model learns to discriminate between positive and negative objects using their feature embeddings. Once trained, the model is essentially applied to the entire data catalog, classifying the data into positive and negative instances (Step ❺). This operation is supported by pre-built index structures to speed up the inference and achieves interactive response times [11]. The user then receives all objects predicted as positive ranked by the model (Step ❻). If the user is not yet satisfied with the results, the iterative fine-tuning phase allows them to continuously refine the search results, thus progressively enhancing the accuracy and relevance (Step ❼).

2.2 CLIP Embeddings

The extraction of appropriate embeddings is crucial for a search engine. Typically, the quality of embeddings improves with larger data sets or, in the case of feature extraction with deep neural networks, with larger model sizes. In the past, this has led to the common practice of adopting pre-trained large-scale models such as CLIP, which are subsequently fine-tuned for specific applications [2].

CLIP-Branches resorts to CLIP embeddings for text and image data. As sketched in Figure 2, CLIP transforms raw image and text data into meaningful embeddings of dimensionality $d = 512$. The extracted features typically capture many of the characteristics of the original data as the model was pre-trained on more than 400 million image-text pairs. However, significant storage is still required. We extend the CLIP model by attaching a custom head model (fully connected neural network) with $d' = 32$ output neurons to reduce these storage requirements and to make the embeddings suitable for decision branches. For the head module to yield meaningful results, it is essential to train the weights of the additional fully connected layers. To achieve this, we employ the MSCOCO Image Captioning (MSCOCO) data set [9] with 82,612 training samples and 40,438 validation samples.[4] To further reduce the storage consumption, we make use of a specialized regularization term, see Section 2.3. This term is specifically designed to favor uniformity in the spherical latent space of the final embedding and plays a crucial role in optimizing the features for the subsequent quantization process.

2.3 Quantization

An essential ingredient of CLIP-Branches is the reduction of the storage requirements of the embeddings. Similar to other approaches, we make use of quantization techniques to achieve this while not harming the expressiveness of the features. To this end, we adopt the so-called Kozachenko-Leononenko (KoLeo) regularization initially introduced by Sablayrolles et al. [16] and later applied in the context of the well-known *DINOv2* model [14]. The regularization encourages the feature vectors to be spread out uniformly across the spherical feature space. This uniform distribution across the embedding space enhances the effectiveness of quantization. Given a batch of n embeddings $x_1, x_2, \ldots, x_n \in \mathbb{R}^{d'}$ in a d'-dimensional space, KoLeo regularization is defined as

$$\mathcal{L}_{KoLeo} = -\frac{1}{n}\sum_{i=1}^{n}\log(\rho_{n,i}), \tag{1}$$

where $\rho_{n,i} = min_{j\neq i}\left\|x_i - x_j\right\|$ is the minimal Euclidean distance between x_i and any other point in the batch. For the quantization itself, we use an 8-bit scalar quantization method by mapping each value to one of 256 integer values. In total, the embedding size is reduced by a factor of 64 (more precisely, from 512 values with single floating point precision to 32 values with a 8-bit representation).

2.4 System Setup

Before deployment, CLIP-Branches undergoes some pre-processing. This involves transforming the image data into quantized CLIP feature embeddings (see Figure 2). Through this process, we can significantly reduce the storage footprint of the raw image data. For instance, the largest data set, the LAION data set, was reduced from 8 TB of image data to just 8 GB of 8-bit feature embeddings. We then construct the required index structures for the initial search and for subsequent refinement using the decision branch models. In total, these index structures sum up to 62 GB of additional storage.

3 EXPERIMENTAL EVALUATION

To assess the benefits of our fine-tuning process over standard text-image retrieval using NN search, we performed evaluations across various image classification data sets with our CLIP embeddings. We compared when the F_1-score of our decision branches models exceeded the score of a classification based on NN search[5] with an increasing number of positive training samples for our models (see Figure 3). In the context of our search engine, we could thereby show that on average 22 positive samples are sufficient to outperform initial search results (based on NN search) w.r.t. the F_1-score with single decision branches. In comparison, only 8 positive samples are required when using decision branch ensembles. Note that the user interface, see Section 4, allows to select such instances quickly.

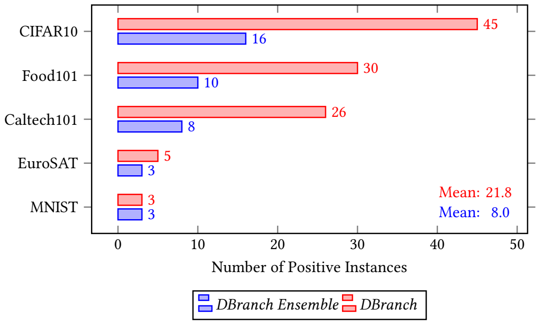


Figure 3: Number of positive training instances required for Decision Branches models to surpass NN search results among different image classification benchmark data sets.

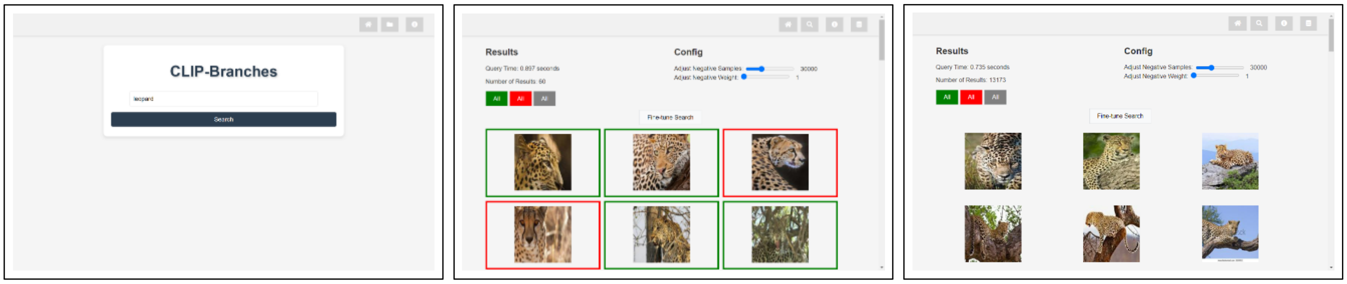
	MSCOCO			Accuracy
Recall@	1	10	100	
CLIP (d = 512)	-	-	-	0.474
CLIP + PCA	0.683	0.757	0.802	0.144
CLIP + Head	0.831	0.890	0.926	0.390
CLIP + Head + KoLeo	0.953	0.970	0.985	0.404

Table 1: Comparison of quantization methods applied to original CLIP features (d = 512) for the MSCOCO data set, detailing recall scores at 1, 10, and 100 nearest neighbors post-quantization, alongside zero-shot classification accuracy based on the embeddings. Each quantization method reduces the feature dimensionality to d = 32.

To assess the impact of the quantization on the quality of the induced features, we resort to the *Recall@k* metric as shown in Table 1. This involves comparing the quality of a set of nearest neighbors computed on one of the quantized features (such as CLIP + PCA followed by quantization), given the set of nearest neighbors computed on the same set of features without quantization (e.g. CLIP + PCA). A Recall@k value close to one indicates a good overlap, indicating minimal impact from quantization on the corresponding feature set. Note that we only report Recall@k values for the reduced feature sets. Table 1 illustrates that the features processed with our head and KoLeo regularization maintain neighborhood structures more effectively through the quantization step than those processed by traditional methods (e.g., CLIP + PCA) or those without KoLeo regularization (CLIP + Head).

Finally, the zero-shot classification results (Accuracy) shown in Table 1 indicate that employing the final CLIP + Head + KoLeo features yields the best accuracy, indicating that these embeddings retain a significant portion of the original information compared to the original CLIP features.

[4]During this training phase, we freeze all the weights of the original CLIP model to exclusively fine-tune the newly added layers. The training was conducted over 100 epochs using a *NVIDIA A100* GPU, under the same settings as the original CLIP model.
[5]All k nearest neighbors found in the test set given a positive training instance are considered to be positive, while all other instances are considered to be negative. The number of k is set to the true number of positives in the test set. Note that such important information is not known in practice and gives the NN search an advantage.



(1) Search Interface **(2) First Top k-Results** **(3) Fine-tuned Results**

Figure 4: Demonstration of CLIP-Branches search workflow. The user initiates a search with a query string and receives top k-initial results. These results are then labeled as positive (green) or negative (red) based on user preference, guiding the fine-tuning of the search. The fine-tuned results reflect more completeness and higher accuracy.

4 DEMONSTRATION

In our demonstration, we show how users interact with our first prototype of CLIP-Branches. To effectively demonstrate its functionality, we have prepared multiple image data sets:

- **CIFAR10**: a subset of the CIFAR10 benchmark data set consisting of 50,000 images.
- **Shutterstock**: a data set collected from the Shutterstock platform[6] consisting of more than 14 million images [13].
- **LAION**: a subset of the LAION-400M web-scraped data set with more than 260 million images [17].

The user can choose between various classification models for refining the search. In addition to decision branch models, we have also incorporated classical decision trees and random forests to highlight the differences in query time when using unoptimized models for the search: (1) a single *decision branch* model, (2) a *decision branch ensemble* comprising 25 individual models, (3) a standard *decision tree* and (4) a *random forest* model. The general workflow and the interface are depicted in Figures 1 and 4, respectively.

(1) *Initiating the search:* When entering the search engine, see Figure 4(1), users first encounter a text search interface. The top bar includes options to choose the data set (accessible via the "*Folder*" icon) and a manual (via the "*Info*" button). Users can input their search string into a dedicated text field and start the search via the respective button. Upon initiating the search, this string is converted into a CLIP feature representation for which the nearest neighbor embeddings are retrieved. To address the computational challenges associated with high-dimensional searches we resort to an approximate nearest neighbor (ANN) search strategy supported by an index[7] for the first search. This method significantly accelerates the retrieval process by approximating the closest neighbors rather than computing exact matches.

(2) *First results and interactive labeling:* After initiating the search, users are presented with an initial set of results, comprising up to 60 images that match their query, see Figure 4(2). Users can interact with these results by labeling images as positive (green frame), negative (red frame), or leaving them unlabeled (no frame). By using the "*All*" buttons, users can simultaneously apply the corresponding label to all displayed images. In the top bar, users can select their preferred fine-tuning classification model via the "*Lens*"-button. In addition, users can tweak search parameters to refine the outcome. A key feature here is the adjustable slider for negative sample inclusion. By increasing the count of randomly sampled negative images, the precision of the model's results is enhanced, but at the expense of longer training times.[8] Another adjustable parameter is the negative weight slider, which determines the influence of user-labeled negative samples relative to randomly chosen ones. Once satisfied with the labeling and parameter configuration, the user can proceed to the next phase by clicking the "*Fine-tune Search*" button.

(3) *Fine-tune search:* The retrieval time depends on the selected model. In general, for quick searches, single decision branch models are well suited. Conversely, for higher accuracy, but with slightly longer wait times, decision branch ensemble models are preferable. For each run, search statistics are shown, see Figure 4(3). If the initial fine-tuned results are not satisfactory, users can iterate the fine-tuning process until the desired outcome is achieved.

Users are invited to explore CLIP-Branches through the provided link: https://web.clip-branches.net/.

5 CONCLUSION

We introduce CLIP-Branches, a text-image search engine that incorporates a novel refinement feature. Both the code for setting up the search engine as well as a video showcasing how users can interact with CLIP-Branches are available on GitHub.[9]

ACKNOWLEDGMENTS

This research is supported by the Independent Research Fund Denmark (grant number 9131-00110B). We also acknowledge support from NVIDIA for a hardware donation.

[6]https://shutterstock.com
[7]We use an implementation of a k-d tree as ANN index [4].

[8]Random sampling assumes that negative instances far outnumber positive ones in the addressed tasks. When positive instances are falsely included in the negative sample, the machine learning models used are generally robust to this small amount of noise.
[9]https://github.com/cluel01/clip-branches

REFERENCES

[1] Jina AI. 2024. CLIP-as-service. https://github.com/jina-ai/clip-as-service.

[2] Josh Beal, Hao-Yu Wu, Dong Huk Park, Andrew Zhai, and Dmitry Kislyuk. 2022. Billion-Scale Pretraining with Vision Transformers for Multi-Task Visual Representations. In *2022 IEEE/CVF Winter Conference on Applications of Computer Vision, WACV*. 1431–1440.

[3] Romain Beaumont. 2022. Clip Retrieval: Easily compute clip embeddings and build a clip retrieval system with them. https://github.com/rom1504/clip-retrieval.

[4] J. L. Bentley. 1975. Multidimensional Binary Search Trees Used For Associative Searching. *Commun. ACM* 18, 9 (1975), 509–517.

[5] Weijing Chen, Linli Yao, and Qin Jin. 2023. Rethinking Benchmarks for Cross-modal Image-text Retrieval. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1241–1251.

[6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. arXiv:2401.08281

[7] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR*. 2946–2953.

[8] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 117–128.

[9] Tsung-Yi Lin, Michael Maire, Serge J. Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision, ECCV*. 740–755.

[10] Tie-Yan Liu. 2009. Learning to Rank for Information Retrieval. *Foundations and Trends in Information Retrieval* 3, 3 (2009), 225–331.

[11] Christian Lülf, Denis Mayr Lima Martins, Marcos Antonio Vaz Salles, Yongluan Zhou, and Fabian Gieseke. 2023. Fast Search-by-Classification for Large-Scale Databases Using Index-Aware Decision Trees and Random Forests. In *Proceedings of the VLDB Endowment*. 2845–2857.

[12] Christian Lülf, Denis Mayr Lima Martins, Marcos Antonio Vaz Salles, Yongluan Zhou, and Fabian Gieseke. 2023. RapidEarth: A Search-by-Classification Engine for Large-Scale Geospatial Imagery. In *Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems, SIGSPATIAL*.

[13] Thao Nguyen, Gabriel Ilharco, Mitchell Wortsman, Sewoong Oh, and Ludwig Schmidt. 2022. Quality Not Quantity: On the Interaction between Dataset Design and Robustness of CLIP. In *Advances in Neural Information Processing Systems, NeurIPS*. 21455–21469.

[14] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. 2023. DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193

[15] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning, ICML*. 8748–8763.

[16] Alexandre Sablayrolles, Matthijs Douze, Cordelia Schmid, and Hervé Jégou. 2019. Spreading vectors for similarity search. In *International Conference on Learning Representations, ICLR*.

[17] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. 2022. Laion-5b: An open large-scale dataset for training next generation image-text models. In *Advances in Neural Information Processing Systems, NeurIPS*. 25278–25294.

[18] Andrew Zhai, Dmitry Kislyuk, Yushi Jing, Michael Feng, Eric Tzeng, Jeff Donahue, Yue Li Du, and Trevor Darrell. 2017. Visual Discovery at Pinterest. In *Proceedings of the 26th International Conference on World Wide Web Companion, WWW*. 515–524.